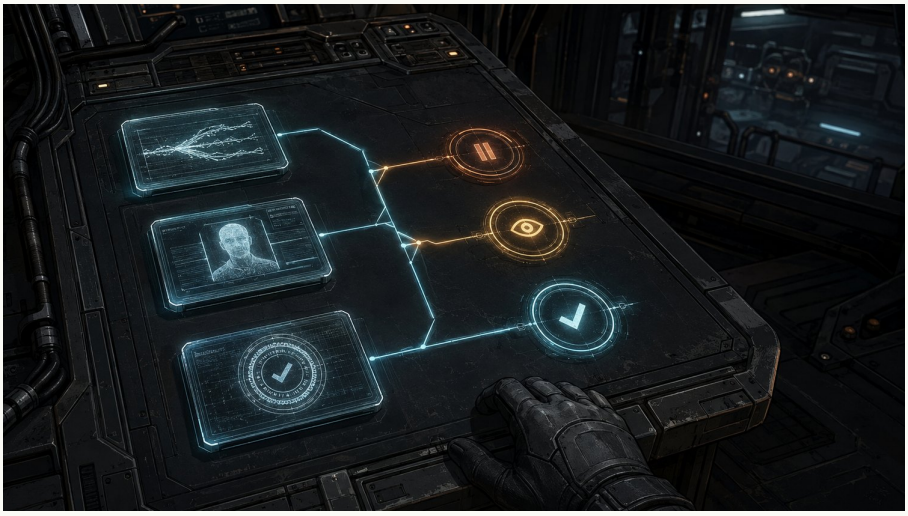
ACTION 9B / 15 POINTS / SAME-DAY CLIENT JOB

The relay file desk

Text rows -> validated records -> a trustworthy report



A complete C# WPF app, from a blank Visual Studio project. Build with the instructor. Keep the evidence as you work.

01	02	03	04
Story + prototype Visual hat	Flow + decisions Logical hat	Types + records Data hat	Test + handoff Git + receipt

9A (10) + 9B (15) = Week 9 (25). Two submissions; no third graded weekly assignment. Canvas controls official dates and times.

No project ZIP. The code is inside this book, with exact placement, explanation and checkpoints. Use the online edition for one-chapter navigation and exact Copy code buttons.

Your lecture / learning route

00 / Before the client arrives

All hats | project, solution, local vs remote, Git

01 / The hire: story becomes a small product

Visual -> Logical -> Data | audience, need, actions, variables, prototype, flowchart

02 / Choose a container, not a bigger problem

Data | JSON, serialization, SQL, SQLite, MySQL, privacy

03 / Start with nothing: create the WPF project

All hats | WPF, XAML, class, namespace, Git

04 / Give the data a shape

Data | class, object, property, type, List<T>, method, return

05 / Build the visible window

Visual | Grid, DockPanel, Menu, x:Name, binding, Click event

06 / Connect the window before adding behavior

Logical | partial class, constructor, scope, event handler, field

07 / Open, parse and review the file

Logical + Data | OpenFileDialog, ReadAllLinesAsync, arrays, for, Split, TryParse, validation

08 / Filter, save and finish the client job

All hats | Boolean filter, method calls, SaveFileDialog, WriteAllTextAsync, output

09 / Ask, test, explain: the class work

Logical + Data | edge cases, debugging, AI verification, peer review

10 / Hand off 9B / 15 points

All hats | documentation, GitHub, receipt, handoff

Recap: show the path from one visible user action to its event, method, data and saved output.

9B / CHAPTER 00 / All hats

Before the client arrives

Open this Action at the start of class. Build with the instructor; pause at each checkpoint. This is one complete client job due today, not half an app.

1. Use Visual Studio on Windows (not VS Code). In Visual Studio Installer > Modify, check .NET desktop development. Use a supported .NET SDK installed in your classroom. The reference build uses .NET 8; .NET 10 is also suitable.

2. Create a local folder such as D:\CS077\Week09\RelayManifest. If D: is unavailable, choose another approved drive with space. Keep generated work off a nearly full system drive.

3. Check GitHub sign-in and repository access. A free account is enough. Open Discord and a notes document. Do not put passwords, tokens, classmates' private data or browser profiles in Git.

4. Keep Evidence/brief.md, prototype.png, whiteboard.jpg, notes.md, tests.md and receipt together beside the project. These are sections/files in one handoff, not extra reports.

5. Need setup help or an accommodation? Tell the instructor now. In-class whiteboard and peer work are required; only the instructor can approve an alternative. Canvas owns official deadlines.

PICTURE IT

A solution is the worksite. A project is one building on it. A local Git commit is a dated checkpoint; pushing copies that history to GitHub.

RUN / CHECK

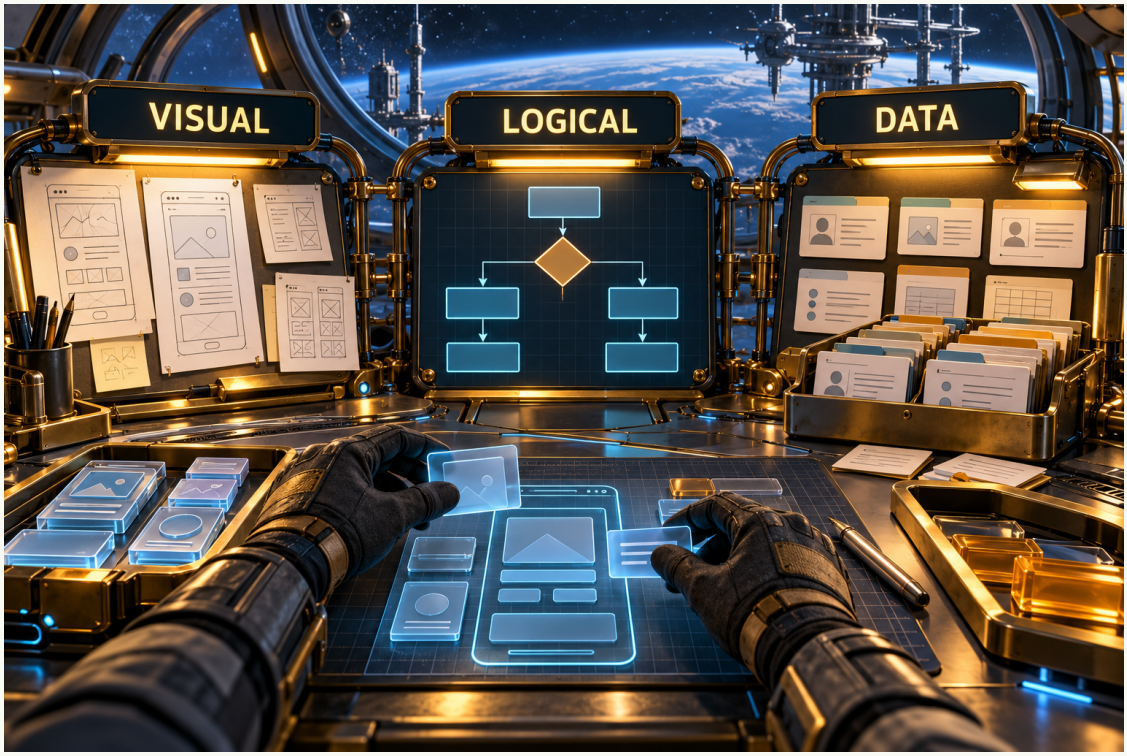
Run a new empty WPF window and confirm you can save and reach your repository. Setup checks are not extra points.

My observation / question / next step

9B / CHAPTER 01 / Visual -> Logical -> Data

The hire: story becomes a small product

A relay supply clerk receives a text manifest that is hard to read. You are hired to open it, flag bad rows and produce a clear summary. This is a new job, not a rewrite of DispatchBoard.



Three hats, one client need. This is an analogy illustration, not a working app or an extra story requirement.

Pause and point

Name one decision in each part of the illustration. Then follow the numbered instructions on the next page.

9B / CHAPTER 01 / GUIDED WORK

The hire: story becomes a small product

1. Story flow: choose the client -> describe the situation -> name the intended user -> name the one problem -> state the smallest useful output. Keep it to five sentences. You may adapt the setting, not remove the required behavior.

2. Actions: open, split, validate, display, filter, export. Containers: one record class; arrays of lines/fields; a list of accepted records. Variables: name (string), qty (int), priority (string), highOnly (bool).

3. Visual hat first: sketch one window in PowerPoint, a photo editor, on paper or with AI. Put the main action first, show feedback and leave room for readable data. Save an image. You do not need decorative images inside the app.

4. Logical hat next: draw your click -> processing -> decision -> result flow on the whiteboard with the class. Copy it in the free browser tool draw.io: choose Device / Decide later, Blank diagram, General/Flowchart, then rectangle (action), diamond (decision), arrows. Save the .drawio locally and export PNG. Paper is an equivalent fallback.

5. Data hat connects the two: write each input and its type, the class/list that holds it, validation rules and the output file. Revisit the Visual hat after a test, but label which decision you are making.

6. Attention First / ADD P2P: explain who needs attention, what they need to notice first and how your feedback prevents confusion. Use your course book as a design reference; this job adds no unannounced chapter/page assignment.

PICTURE IT

Wear one hat to make one decision. The Visual hat designs the counter; the Logical hat defines the service; the Data hat labels what passes across it. They cooperate, but they are not the same job.

RUN / CHECK

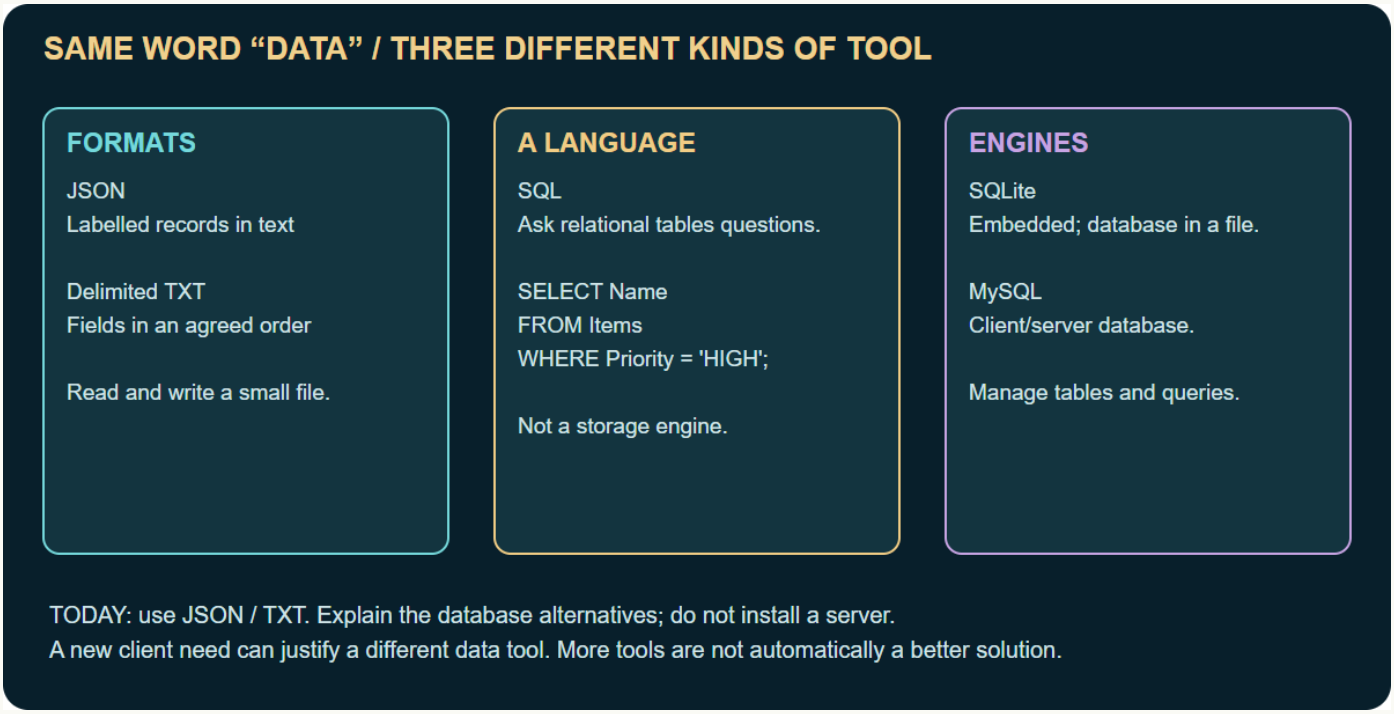
Save the brief, prototype and one annotated whiteboard/flowchart. Ask a peer to follow the arrows before you code.

My observation / question / next step

9B / CHAPTER 02 / Data

Choose a container, not a bigger problem

JSON and TXT are ways to represent data, not database engines. SQL is a language for asking relational databases questions. SQLite is an embedded engine that stores a database in a file. MySQL is a client/server database product. Today we build file/JSON apps, compare these options and write a choice note; no SQL server, database package or paid service is required.



Choose the smallest tool that meets the need. These projects do not require a database installation.

Pause and point

Name one decision in each part of the illustration. Then follow the numbered instructions on the next page.

9B / CHAPTER 02 / GUIDED WORK

Choose a container, not a bigger problem

1. JSON: labelled records in text. A JSON object uses { }; an array uses [ ]. A string uses double quotes; a Boolean is true/false without quotes. A downloaded response is a snapshot, not a live database connection.

2. Text with delimiters: agree on field order and a separator. Our format uses name|quantity|priority. It is simple, but names must not contain the | separator. This is not a general CSV reader.

3. SQL: SELECT Name FROM Items WHERE Priority = 'HIGH'; asks for rows. Read it as: choose the Name column from the Items table where Priority is HIGH. This is an illustration only; our apps do not execute SQL.

4. SQLite: useful when a single-device app needs tables, durable edits and structured queries. MySQL: useful when multiple clients need a shared, administered database. Neither choice automatically supplies security, backups or a good schema.

5. Write two sentences: why today's format is enough, and one change in client need that would justify a database. A server needs authorization; never place database passwords in a desktop app or public repo.

PICTURE IT

JSON is a labelled delivery box. A text row is a form with fixed spaces. A database is an organized records room; SQL is the request language. SQLite and MySQL are two different systems for running that room.

RUN / CHECK

Explain the difference aloud. Submit the two-sentence choice note inside your existing notes, not a new report.

My observation / question / next step

9B / CHAPTER 03 / All hats

Start with nothing: create the WPF project

Do not download an app template. Visual Studio creates the empty framework; you add every project-specific control and behavior.

1. Visual Studio > Create a new project. Search WPF. Select WPF Application, language C#, Windows, Desktop. Prefer modern .NET, not WPF App (.NET Framework). Name it RelayManifest, exactly, so the namespaces below match.

2. Set Location to your local Week09 folder. Choose .NET 8 if available, otherwise the supported modern .NET version your instructor uses. Do not manually change the framework unless directed. Select Create.

3. Solution Explorer: App.xaml starts the app; MainWindow.xaml describes the window; MainWindow.xaml.cs is its C# code-behind. Expand the little arrow beside MainWindow.xaml to find the .cs file.

4. Press F5 or the green Start button. See the empty window; close it to stop debugging before editing. If .NET desktop development or WPF Application is missing, return to the setup check.

5. Git > Create Git Repository. Choose the RelayManifest solution folder (the folder containing your .sln or .slnx). Create a GitHub repository with access the instructor can use. Check .gitignore excludes .vs/, bin/ and obj/.

6. Git Changes: message "Create blank WPF project" > Commit All > Push (or Publish). At GitHub confirm App.xaml, MainWindow.xaml and the .csproj exist. A private repo needs instructor access; do not make private class evidence public without permission.

PICTURE IT

XAML is the floor plan. C# is what the occupants do. x:Class is the address joining that plan to its code-behind class.

RUN / CHECK

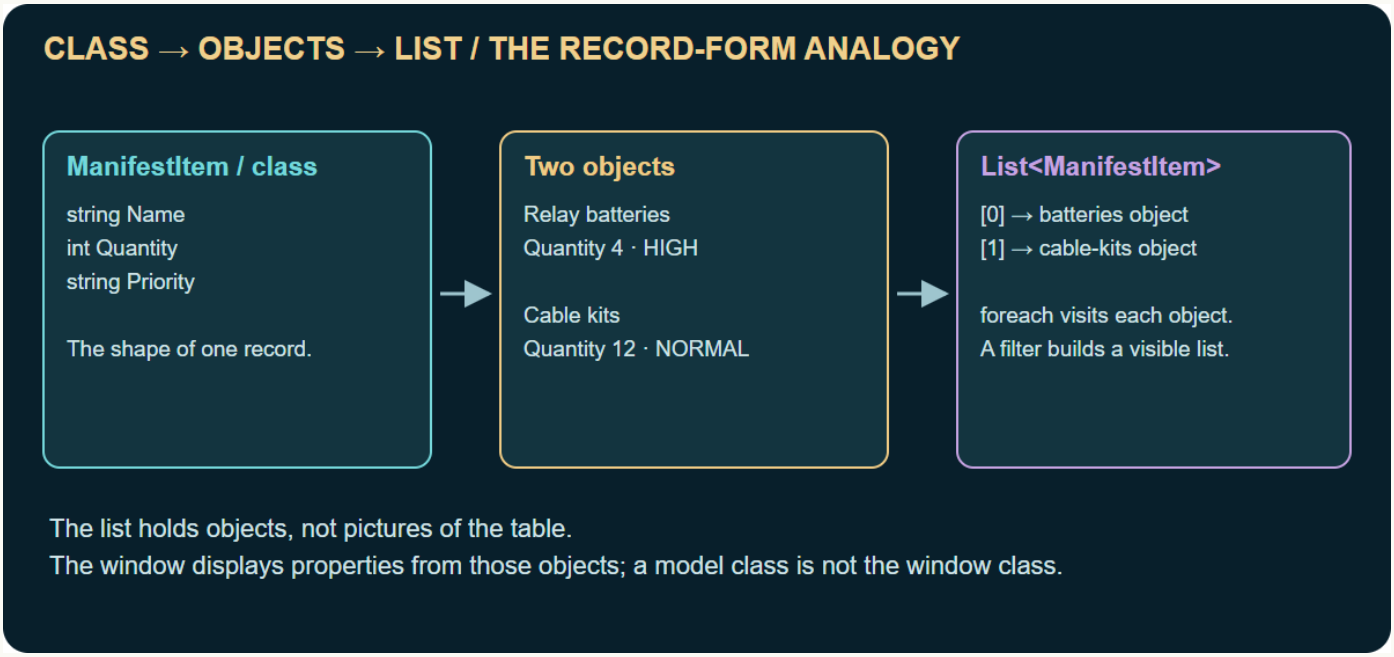
Your own blank project runs. The first Git checkpoint exists remotely. Do not paste code into the Output panel or the Designer surface.

My observation / question / next step

9B / CHAPTER 04 / Data

Give the data a shape

A class describes the fields one record can have. An object is one actual record. A List<T> holds many objects of that type.



Class = the blank form. Object = a filled form. List = the tray. Control names belong to the window, not these forms.

1. Right-click the RelayManifest project in Solution Explorer > Add > Class. Name it ManifestItem.cs. Open the new file. Replace ALL its contents with the first block.

2. Add another class file named ManifestData.cs. Replace ALL its contents with the second block. Do not paste these classes inside MainWindow or a click method.

3. Read each method header before the body: its name, parameters (inputs) and return type (output). static means we call the helper on the class itself. var lets the compiler infer the type; it does not mean the value has no type.

4. Parse receives a string[] array of lines. The for loop visits each index; Split creates another array of fields. TryParse checks the number without crashing. continue skips the current bad row, not the whole file.

5. Select makes a new visible list without deleting the original records. Summary turns that list into output text. This separates data logic from the window so we can test it without clicking a UI.

9B / CHAPTER 04 / CODE

One manifest record

WHERE: Replace ALL of ManifestItem.cs.

```
namespace RelayManifest;

public class ManifestItem
{
    public string Name { get; set; } = "";
    public int Quantity { get; set; }
    public string Priority { get; set; } = "NORMAL";
}

public class ParseResult
{
    public List<ManifestItem> Items { get; } = new();
    public List<string> Warnings { get; } = new();
}
```

What this code means

get; set;: Read or change a property. The empty string is a safe initial value, not null.

bool / int / string: True-or-false; a whole number; text. Do not treat quantity as text when you need arithmetic.

9B / CHAPTER 04 / CODE

Data work, away from the window

WHERE: Replace ALL of ManifestData.cs.

```
namespace RelayManifest;

public static class ManifestData
{
    public static ParseResult Parse(string[] lines)
    {
        var result = new ParseResult();
        for (int index = 0; index < lines.Length; index++)
        {
            string line = lines[index].Trim();
            if (line.Length == 0 || line.StartsWith("#")) continue;
            string[] parts = line.Split('|');
            if (parts.Length != 3)
            {
                result.Warnings.Add($"Line {index + 1}: use 3 fields.");
                continue;
            }
            string name = parts[0].Trim();
            string priority = parts[2].Trim().ToUpperInvariant();
            bool validNumber = int.TryParse(parts[1].Trim(), out int qty);
            if (name.Length == 0 || !validNumber || qty < 1 || qty > 500 ||
                (priority != "HIGH" && priority != "NORMAL"))
            {
                result.Warnings.Add($"Line {index + 1}: check name, " +
                    "quantity 1-500 and HIGH/NORMAL.");
                continue;
            }
            result.Items.Add(new ManifestItem
            {
                Name = name, Quantity = qty, Priority = priority
            });
        }
        return result;
    }

    public static List<ManifestItem> Select(
        List<ManifestItem> items, bool highOnly)
    {
        var visible = new List<ManifestItem>();
        foreach (var item in items)
```

9B / CHAPTER 04 / CODE CONTINUED

Data work, away from the window

WHERE: Replace ALL of ManifestData.cs.

```
            if (!highOnly || item.Priority == "HIGH") visible.Add(item);
        return visible;
    }

    public static string Summary(List<ManifestItem> items)
    {
        var lines = new List<string> { "RELAY MANIFEST SUMMARY" };
        int total = 0;
        foreach (var item in items)
        {
            total += item.Quantity;
            lines.Add($"{item.Name} | {item.Quantity} | {item.Priority}");
        }
        lines.Add($"Total visible units: {total}");
        return string.Join(Environment.NewLine, lines);
    }
}
```

What this code means

int.TryParse(..., out int qty): Returns success/failure and puts the parsed number in qty. Both matter.

index + 1: Array indexes start at zero; a human file line number starts at one.

ParseResult: Returns both accepted items and warnings instead of hiding bad data.

PICTURE IT

A class is a blank record form; an object is a filled-in form. The list is the tray holding the forms. A method is a labelled work box: inputs go in, steps happen inside { }, return hands a result back.

RUN / CHECK

Build > Build Solution (Ctrl+Shift+B). These new files compile before you connect the window.

9B / CHAPTER 05 / Visual

Build the visible window

Build the window in five visible blocks. Predict the result, add only the named block, run it, then explain what changed. Use the complete file only for comparison or recovery.



Anatomy map: nesting means ownership; row/dock properties mean placement. Follow the matching block number.

1. Use the guided blocks below while building with the instructor. They produce the same final interface; they do not add another assignment.

2. Keep MainWindow.xaml and MainWindow.xaml.cs side by side. XAML goes in the .xaml editor; only Block 5 adds a small C# probe inside the existing constructor.

3. After each block, save, build and run. Close the running app before the next edit. A visible control can be correct even when its app behavior is not connected yet.

4. Keep the comment markers until the step tells you to replace them. Do not paste a second Grid or a second Window around an existing one.

5. If a checkpoint fails, compare the matching screenshot and error first. Keep the last working Git checkpoint before asking AI for one small change.

PICTURE IT

Build a room before installing its shelves; place the controls before connecting their behavior; test one record before trusting a whole file. Each run answers one question.

9B / GUIDED CONSTRUCTION / BLOCK 1

The outer frame



PREDICT: What can a window show before it has controls?

WHY: Window is the outer container. One layout panel will hold everything inside it.

BLOCK 1 / ADD ONLY THE NAMED PART

Block 1 / a window and one empty layout panel

WHERE: Replace ALL of MainWindow.xaml. Leave MainWindow.xaml.cs as Visual Studio created it.

```
<Window x:Class="RelayManifest.MainWindow"
 xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
 xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
 Title="Relay Manifest | File desk" Width="1000" Height="680"
 MinWidth="780" MinHeight="540" Background="#071B25"
 Foreground="#E5F3F1">
<DockPanel>
  <!-- MENU -->
  <Grid Margin="28">
    <!-- LAYOUT -->
  </Grid>
</DockPanel>
</Window>
```

Nesting: Window has one direct content element: our Grid or DockPanel. Window.Resources is a property element, not another visible child.

Closing tags: The last opened container closes first. Match each opening/closing pair before running.

RUN / EXPLAIN

Build and run: an empty dark window with the correct title. No buttons, data or file behavior yet.

Predict -> add -> run -> explain. Keep the last working checkpoint.

9B / GUIDED CONSTRUCTION / BLOCK 2

Give the window rooms



PREDICT: Which area should stretch when the window gets taller?

WHY: DockPanel reserves the top for a menu. A Grid below it separates title, records, status and warnings.

BLOCK 2 / ADD ONLY THE NAMED PART

Shared styles / one place for repeated decisions

WHERE: Insert immediately after the opening <Window ...> tag, before the layout panel. Do not add a second Window.Resources block.

```
<Window.Resources>
  <Style TargetType="Button">
    <Setter Property="Padding" Value="14,9"/>
    <Setter Property="Margin" Value="0,0,8,0"/>
    <Setter Property="Background" Value="#B9E5E5"/>
    <Setter Property="Foreground" Value="#09212B"/>
  </Style>
  <Style TargetType="TextBlock">
    <Setter Property="TextWrapping" Value="Wrap"/>
  </Style>
</Window.Resources>
```

Style: One reusable set of properties. Button padding affects all buttons; it does not create a button.

BLOCK 2 / ADD ONLY THE NAMED PART

Block 2 / row definitions and labelled spaces

WHERE: Replace ONLY <!-- LAYOUT --> inside the Grid. Keep the surrounding Grid and Window closing tags.

```
<Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="*"/>
      <RowDefinition Height="Auto"/>
      <RowDefinition Height="100"/>
    </Grid.RowDefinitions>
<!-- HEADER -->
<!-- RECORDS -->
```

Auto / * / 100: Auto uses needed space; * shares remaining space; 100 is a fixed device-independent size.

Grid.Row: A child says which numbered row to occupy. Rows start at zero; the default is row zero.

RUN / EXPLAIN

Build and run: still mostly empty. The layout now reserves space. Resize the window; the star row receives the spare height.

Predict -> add -> run -> explain. Keep the last working checkpoint.

9B / GUIDED CONSTRUCTION / BLOCK 3

Place the labels and actions



PREDICT: Will a visible button or menu item work before it has an event handler?

WHY: Content and labels explain what the user can do. A name creates a C# control reference. We deliberately wait to connect Click events.

BLOCK 3 / ADD ONLY THE NAMED PART

Block 3A / the heading

WHERE: Replace ONLY <!-- HEADER --> with this StackPanel.

```
<StackPanel Margin="0,0,0,18">
      <TextBlock Text="CLIENT JOB 9B / RELAY FILE DESK"
       Foreground="#F3B37C" FontSize="13"/>
      <TextBlock Text="Make the manifest readable."
       FontSize="32" Margin="0,8,0,6"/>
      <TextBlock Text="File > Open. Review warnings before sharing."/>
    </StackPanel>
```

StackPanel: Stacks its children in order. Margin adds breathing room; FontSize changes emphasis.

BLOCK 3 / THE TOP MENU

Add the commands after the heading

BLOCK 3 / ADD ONLY THE NAMED PART

Block 3B / visible actions, not behavior yet

WHERE: Replace ONLY <!-- MENU -->, outside the Grid but inside DockPanel. Do not add Click attributes yet.

```
<Menu DockPanel.Dock="Top" Background="#EBCBA6"
   Foreground="#172D34" Padding="12,6">
    <MenuItem Header="_File">
      <MenuItem Header="_Open manifest..."/>
      <MenuItem Header="_Save visible summary..."/>
      <Separator/>
      <MenuItem Header="E_xit"/>
    </MenuItem>
    <MenuItem Header="_View">
      <MenuItem Header="_All records"/>
      <MenuItem Header="_High priority"/>
    </MenuItem>
    <MenuItem Header="_Help">
      <MenuItem Header="_About"/>
    </MenuItem>
  </Menu>
```

x:Name: Names a control so code can find it. It is not the name of a data property.

MenuItem: File, View and Help are siblings across the menu. Their child MenuItems form dropdown commands.

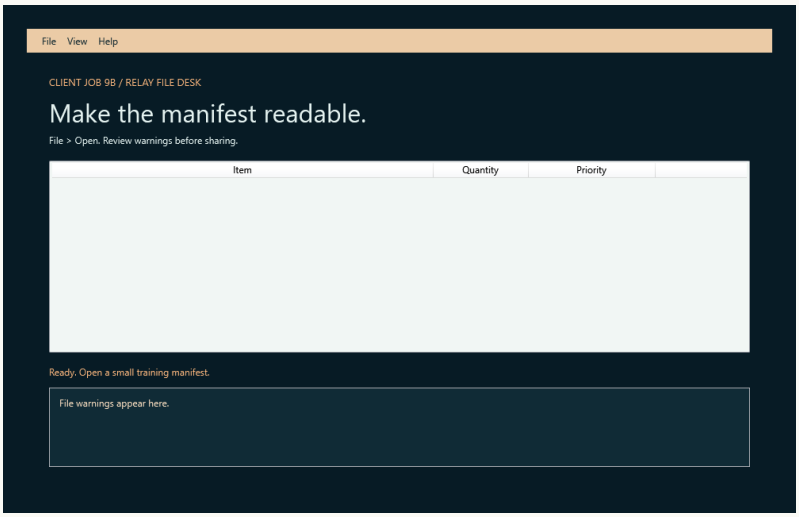
RUN / EXPLAIN

Build and run: headings and actions are visible. Menu dropdowns open; application commands do nothing yet. That is the expected state.

Predict -> add -> run -> explain. Keep the last working checkpoint.

9B / GUIDED CONSTRUCTION / BLOCK 4

Give records a place on screen



PREDICT: Can a column binding display a value before any records are supplied?

WHY: The table describes which property belongs in each column. Status and warnings explain what happened without making the learner inspect code.

BLOCK 4 / ADD ONLY THE NAMED PART

Block 4 / the record table and feedback

WHERE: Replace ONLY <!-- RECORDS --> with this block. The table remains empty until Block 5.

```
<ListView x:Name="lstItems" Grid.Row="1"
     Background="#F1F6F4" Foreground="#152E39">
      <ListView.View><GridView>
        <GridViewColumn Header="Item" Width="480"
         DisplayMemberBinding="{Binding Name}"/>
        <GridViewColumn Header="Quantity" Width="120"
         DisplayMemberBinding="{Binding Quantity}"/>
        <GridViewColumn Header="Priority" Width="160"
         DisplayMemberBinding="{Binding Priority}"/>
      </GridView></ListView.View>
    </ListView>
<TextBlock x:Name="txtStatus" Grid.Row="2"
     Text="Ready. Open a small training manifest."
     Margin="0,16,0,12" Foreground="#F3B37C"/>
<TextBox x:Name="txtWarnings" Grid.Row="3" IsReadOnly="True"
     TextWrapping="Wrap" VerticalScrollBarVisibility="Auto"
     Background="#102B36" Foreground="#F4DBBE"
     Text="File warnings appear here." Padding="10"/>
```

ListView.View / GridView: Defines the columns used for each row. This GridView is not the outer layout Grid.

DisplayMemberBinding: Reads a property on the object for that row. Property spelling and case must match the C# class.

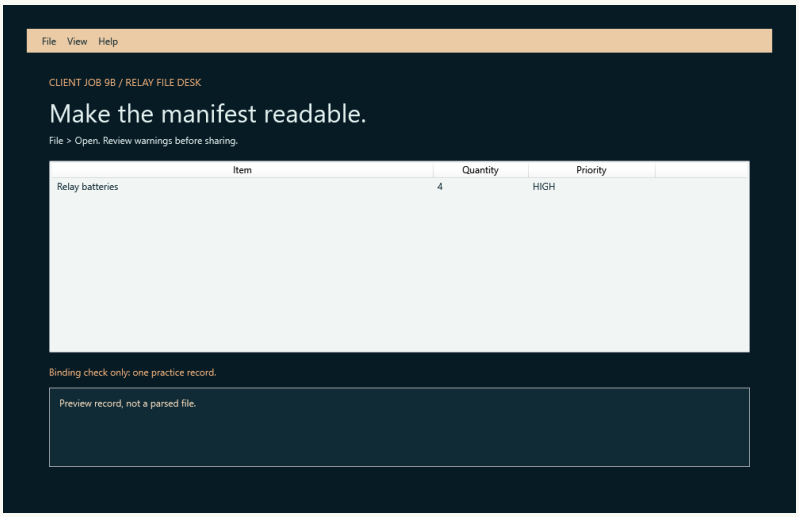
RUN / EXPLAIN

Build and run: an empty table with column headings and visible feedback. Empty is correct: XAML has defined the display, not supplied the records.

Predict -> add -> run -> explain. Keep the last working checkpoint.

9B / GUIDED CONSTRUCTION / BLOCK 5

Prove one binding before loading a file



PREDICT: Which column reads ManifestItem.Quantity, and which control receives the whole list?

WHY: A tiny in-memory record lets you isolate the display from HTTP, files and parsing. Test one relationship before adding several sources of failure.

BLOCK 5 / ADD ONLY THE NAMED PART

Block 5 / temporary binding probe

WHERE: ADD inside the constructor after InitializeComponent(); and before that constructor's closing }. Do not replace the whole file. This is temporary teaching data.

```
// TEMPORARY: inside the constructor, after InitializeComponent().
lstItems.ItemsSource = new List<ManifestItem>
{
    new ManifestItem
    {
        Name = "Relay batteries",
        Quantity = 4,
        Priority = "HIGH"
    }
};
txtStatus.Text = "Binding check only: one practice record.";
txtWarnings.Text = "Preview record, not a parsed file.";
```

ItemsSource: Receives the collection. WPF creates a row display for each object.

Quantity -> {Binding Quantity}: The row binding reads that property on the current record; it does not search by x:Name.

RUN / EXPLAIN

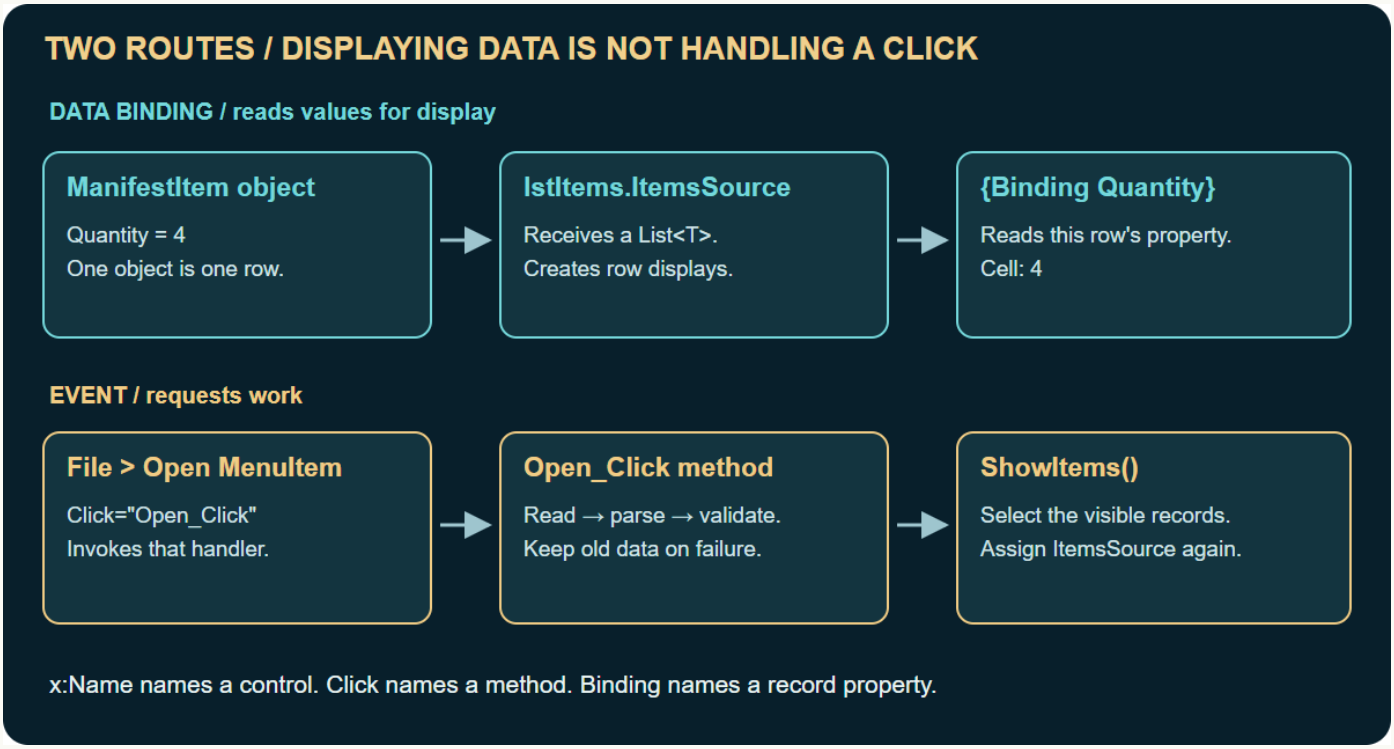
Build and run: Relay batteries / 4 / HIGH. Change Quantity in this probe, rerun and see its cell change. Restore the example.

Predict -> add -> run -> explain. Keep the last working checkpoint.

9B / CHAPTER 06 / Logical

Connect the window before adding behavior

The visible window and one-record binding already work. Replace the temporary probe with the shared fields and event handlers, then connect the Click attributes. This is a separate relationship from data binding.



Two routes: events request work; bindings display data. x:Name, Click and Binding are not interchangeable.

1. Open MainWindow.xaml.cs. Replace ALL of the file with the code block below. This deliberately removes the temporary constructor probe from Block 5. Do not keep a second constructor or a second MainWindow class.

2. A using line imports names; it does not download a package. The namespace matches this project. partial means XAML-generated code and your code share one class.

3. The constructor has the same name as the class and no return type. InitializeComponent creates the controls from XAML before you access them.

4. Fields live inside the class but outside methods. They remember records and the filter between clicks. A local variable inside a method exists only during that method call.

5. Each event handler receives sender and event information, then runs when the matching Click occurs. An event is the notification; the handler is your response.

6. Now return to MainWindow.xaml. Add the Click attributes listed in the connection map below to their existing opening control tags. Do not create duplicate buttons or menu commands. Handler names must match the code-behind exactly.

7. Build and run. Click each button/menu command. The status changes to "connected". Nothing loads or saves yet. Close the app and commit "Window and event wiring".

9B / CHAPTER 06 / CODE

Code-behind checkpoint 1

WHERE: Replace ALL of MainWindow.xaml.cs.

```
using System.IO;
using System.Windows;
using Microsoft.Win32;

namespace RelayManifest;

public partial class MainWindow : Window
{
    private List<ManifestItem> items = new();
    private bool highOnly = false;

    public MainWindow()
    {
        InitializeComponent();
    }

    // DATA METHODS GO HERE

    private void Open_Click(object sender, RoutedEventArgs e)
    {
        txtStatus.Text = "Open connected. Add behavior in the next chapter.";
    }

    private void Save_Click(object sender, RoutedEventArgs e)
    {
        txtStatus.Text = "Save connected. Add behavior in the next chapter.";
    }

    private void All_Click(object sender, RoutedEventArgs e)
    {
        txtStatus.Text = "All connected. Add behavior in the next chapter.";
    }

    private void High_Click(object sender, RoutedEventArgs e)
    {
        txtStatus.Text = "High connected. Add behavior in the next chapter.";
    }

    private void Exit_Click(object sender, RoutedEventArgs e)
    {
```

9B / CHAPTER 06 / CODE CONTINUED

Code-behind checkpoint 1

WHERE: Replace ALL of MainWindow.xaml.cs.

```
        txtStatus.Text = "Exit connected. Add behavior in the next chapter.";
    }

    private void About_Click(object sender, RoutedEventArgs e)
    {
        txtStatus.Text = "About connected. Add behavior in the next chapter.";
    }
}
```

What this code means

private: Only this class uses the member directly.

void: This method hands back no value. Later Parse returns a list instead.

{ }: The outer class braces contain fields and methods. Each inner method has its own pair. Do not nest one method inside another.

CONNECT AFTER THE HANDLERS EXIST

Exact event connections

In each existing Button/MenuItem opening tag, add the attribute before > or />. Do not create a second control.

Visible action	Find control	Add attribute
File > Open manifest	Open manifest MenuItem	Click="Open_Click"
File > Save visible summary	Save visible summary MenuItem	Click="Save_Click"
File > Exit	Exit MenuItem	Click="Exit_Click"
View > All records	All records MenuItem	Click="All_Click"
View > High priority	High priority MenuItem	Click="High_Click"
Help > About	About MenuItem	Click="About_Click"

ItemsSource receives the list; each GridViewColumn reads a property from its current row object. x:Name gives C# a control reference. Click names a method to run. These plain record classes do not automatically notify the UI when an existing property changes. The finished apps call ShowTasks/ShowItems to assign a freshly selected list; automatic live editing would need notification support, which is outside this lesson.

Prediction check: changing a button label does not change a record property. A misspelled binding can leave a blank cell without causing a compile error. Check the C# property and Debug Output.

PICTURE IT

The class is the workshop. Fields sit on shared shelves. A method is one bench. A local variable stays on that bench; it is not automatically visible on another one.

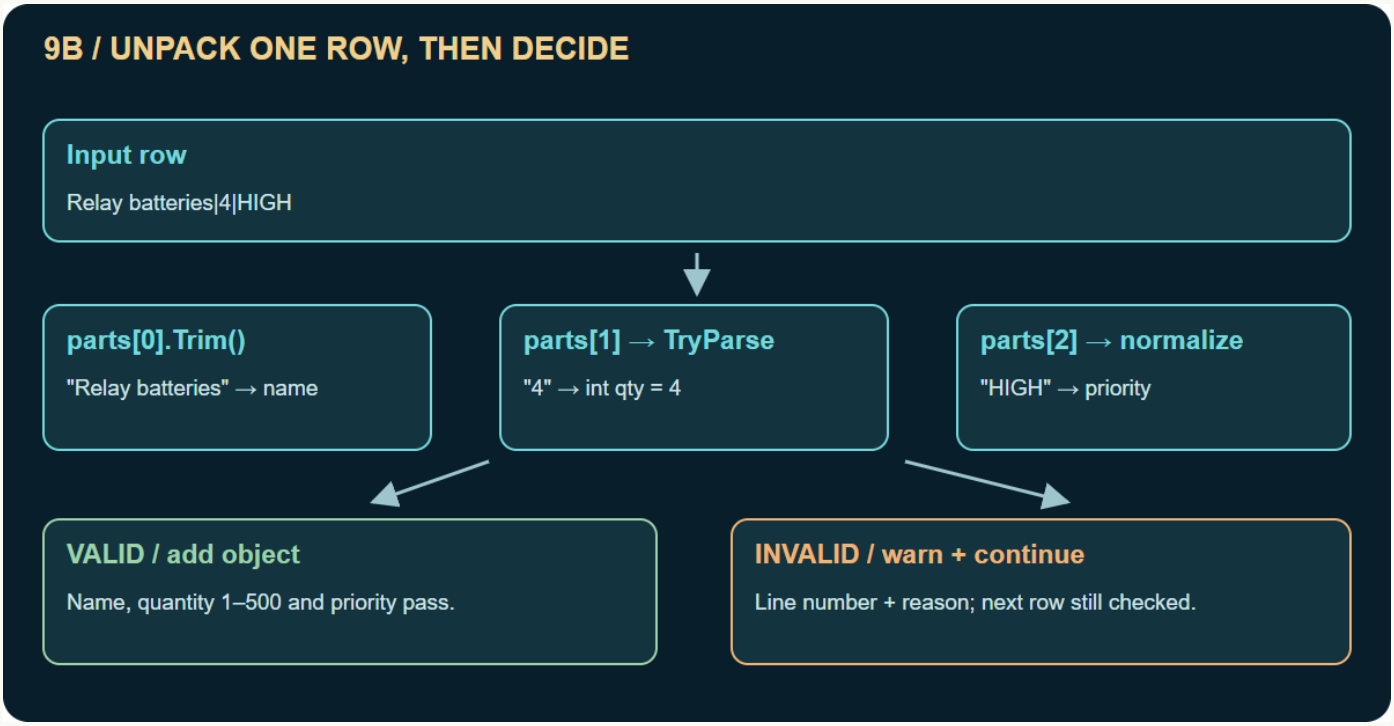
RUN / CHECK

Every event is connected without compilation errors. This is checkpoint 1: a real UI with intentional temporary handlers.

9B / CHAPTER 07 / Logical + Data

Open, parse and review the file

The menu opens a small text file. A successful read is not proof of valid data: each non-comment row must pass the format rules.



Inspect the input, unpack its shape, validate it, then display it. A read succeeding does not prove the data is valid.

1. Download relay-manifest.txt and manifest-with-errors.txt to SampleData/. The format is exactly name|quantity|priority. Quantity is 1-500. Priority is HIGH or NORMAL (case is normalized). # starts a comment. Blank lines are ignored.

2. In MainWindow.xaml.cs, find // DATA METHODS GO HERE. Replace that comment with the first block below. This code is inside the MainWindow class but outside the constructor and event methods.

3. Find the existing Open_Click method. Select its header and entire { ... } body, then replace it with the second block. Do not paste the new method inside the old one.

4. async/await lets the window stay responsive while the read waits. async void is appropriate for these UI event handlers; helpers that return data do not use async void.

5. ReadAllLinesAsync returns a string[] array. ParseResult returns valid records and warnings together. Bad rows are skipped with their line numbers. If no rows pass, the previous list remains visible and the status says so.

6. Cancel the file picker: return ends the handler quietly. try runs the risky read; catch handles expected failures with a visible message. Do not remove the catch just because the happy path works.

7. Build/run: File > Open manifest. The clean sample has 3 rows. The error sample has 2 accepted rows and 3 warnings. Read the warnings before trusting a total.

9B / CHAPTER 07 / CODE

Display helper

WHERE: Replace // DATA METHODS GO HERE inside the class.

```
    private void ShowItems()
    {
        var visible = ManifestData.Select(items, highOnly);
        lstItems.ItemsSource = visible;
        txtStatus.Text = $"{visible.Count} of {items.Count} records | " +
            (highOnly ? "High priority" : "All records");
    }
```

What this code means

ItemsSource: Give the table the visible list; the original list stays in the field.

Method call: ShowTasks() or ShowItems() reuses the same display behavior after every change.

9B / CHAPTER 07 / CODE

Replace the temporary load handler

WHERE: Replace the entire Open_Click method.

```
    private async void Open_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog
        {
            Filter = "Text files (*.txt)|*.txt"
        };
        if (dialog.ShowDialog() != true) return;
        try
        {
            if (new FileInfo(dialog.FileName).Length > 65536)
                throw new InvalidDataException("Use a file below 64 KB.");
            string[] lines = await File.ReadAllLinesAsync(dialog.FileName);
            ParseResult parsed = ManifestData.Parse(lines);
            txtWarnings.Text = parsed.Warnings.Count == 0
                ? "No skipped lines."
                : string.Join(Environment.NewLine, parsed.Warnings);
            if (parsed.Items.Count == 0)
            {
                txtStatus.Text = "No valid rows. Previous list kept.";
                return;
            }
            items = parsed.Items;
            ShowItems();
        }
        catch (Exception ex) when (ex is IOException ||
            ex is UnauthorizedAccessException || ex is InvalidDataException)
        {
            txtStatus.Text = "File not loaded. " + ex.Message;
        }
    }
```

What this code means

return: Exit early on cancel or no valid data.

FileInfo.Length: Check the local input size before reading (256 KB JSON; 64 KB manifest).

catch filter: Only expected read/data errors become user messages; programming errors should still be investigated.

PICTURE IT

Parsing is unpacking a form: Split finds the boxes; Trim removes padding; TryParse checks the number; the condition accepts or rejects the row. A warning is a visible reason, not a hidden failure.

RUN / CHECK

This is checkpoint 2. Record the loaded rows and at least one failed read/invalid-data message. Git checkpoint: "Load and validate data".

9B / CHAPTER 08 / All hats

Filter, save and finish the client job

The client needs a useful result, not just a window that opens. Finish the remaining event methods and test the exported file.

1. Replace the ENTIRE temporary Save_Click, All_Click, High_Click, Exit_Click and About_Click methods with the block below. Keep your working Open_Click and ShowItems unchanged.

2. SaveFileDialog asks the user for an output path. Save inside your own project Evidence/Outputs folder, using a new filename. Confirm overwrite prompts only when you intend to replace a previous summary; never choose the source file.

3. WriteAllTextAsync writes the string returned by Summary. A summary is a separate report file, not a database update and not a reloadable copy of the original format.

4. Open the clean sample: All = 3. View > High priority = 2. Save a summary: total visible units = 7. View > All restores 3. File > Exit closes; Help > About explains the format.

5. No data? Save shows a message, not an empty misleading report. Cancel? No new file. Save denied or wrong path? A visible message; no claim of success.

6. Polish one thing that helps the user: clearer status, spacing, label or contrast. Do not add unrelated features. Capture the final interface and the actual output file. Commit and push.

9B / CHAPTER 08 / CODE

Finish the remaining actions

WHERE: Replace the temporary methods named above. All methods remain inside MainWindow.

```
    private void All_Click(object sender, RoutedEventArgs e)
    {
        highOnly = false;
        ShowItems();
    }

    private void High_Click(object sender, RoutedEventArgs e)
    {
        highOnly = true;
        ShowItems();
    }

    private async void Save_Click(object sender, RoutedEventArgs e)
    {
        var visible = ManifestData.Select(items, highOnly);
        if (visible.Count == 0)
        {
            txtStatus.Text = "No visible records to save.";
            return;
        }
        var dialog = new SaveFileDialog
        {
            Filter = "Text files (*.txt)|*.txt",
            FileName = "manifest-summary.txt"
        };
        if (dialog.ShowDialog() != true) return;
        try
        {
            await File.WriteAllTextAsync(dialog.FileName,
                ManifestData.Summary(visible));
            txtStatus.Text = "Saved visible records. Original untouched.";
        }
        catch (Exception ex) when (ex is IOException ||
            ex is UnauthorizedAccessException)
        {
            txtStatus.Text = "Summary not saved. " + ex.Message;
        }
    }

    private void Exit_Click(object sender, RoutedEventArgs e)
```

9B / CHAPTER 08 / CODE CONTINUED

Finish the remaining actions

WHERE: Replace the temporary methods named above. All methods remain inside MainWindow.

```
    {
        Close();
    }

    private void About_Click(object sender, RoutedEventArgs e)
    {
        MessageBox.Show("Training manifest: name|quantity|priority. " +
            "Use fictional data; no private information.", "About");
    }
```

What this code means

Select -> Summary -> Write: Choose records, make text, then write the file. Three different responsibilities.

Save vs commit vs push: Save writes a file. Commit records project history locally. Push sends commits to GitHub. You need all three.

PICTURE IT

Filtering is a viewing lens, not a shredder. Saving the visible result is making a report from the lens; the original records stay in their tray.

RUN / CHECK

Checkpoint 3: the whole app compiles and the complete client workflow works from start to finish.

9B / CHAPTER 09 / Logical + Data

Ask, test, explain: the class work

Use fictional data only. Test deliberately, explain the result, and keep the evidence you made in class.

1. Test clean input; 2-valid/3-invalid sample; a non-number; missing field; bad priority; an all-invalid file; blank/comment lines; file-picker cancel; save before load; HIGH filter and exported total. Use a copy of the sample.

2. In Evidence/tests.md use: Input -> Expected -> Observed -> Next step. Screenshots show loaded data, one validation/warning state and a saved result. Do not substitute a screenshot of code for the running app.

3. Ask AI one precise question using the prompt below. Read the suggested file/scope before pasting. Test it and record what you kept, changed or rejected. If live AI is unavailable or you cannot use an account, use the supplied response below with the instructor. No paid or personal account is required.

4. Whiteboard recap with a partner: point from a visible action to its event, method, data and output. Each person explains one part. Post one concise Discord peer check: tested action, observed result, unresolved question. Use the approved alternate route if Discord is unavailable.

5. Keep a short TODO for anything unfinished. Show the real state and next step; do not label a failing app complete. Submit today's work by the Canvas deadline and ask the instructor about recovery; required behavior is still assessed.

PICTURE IT

AI is a proposed patch from a teammate, not a passed test. Your job is to explain where it goes and observe whether it works.

Ask AI / one useful unknown

I am a beginner building RelayManifest in C# WPF. Explain string.Split, int.TryParse and continue using one name|quantity|priority row. Name the exact file/method for parsing, show one valid and one invalid input, and give expected results. Do not rewrite my app. I will test your explanation.

No account or live AI? Use this supplied response, test it and record your decision.

Supplied AI-response alternative: in ManifestData.Parse, Split('|') creates three text fields only if the row has two separators. TryParse answers whether the middle field is a number and stores it in qty. If it fails, add a line-number warning and continue to the next row. Test Cells|2|HIGH (accepted) and Cells|many|HIGH (rejected with a warning). Reject any advice that silently changes many into zero.

RUN / CHECK

Save your AI question/response summary, test and decision in notes.md. Reuse your whiteboard and Discord record; no duplicate report.

My observation / question / next step

9B / CHAPTER 10 / All hats

Hand off 9B / 15 points

This job is due the same day you build it. The two Action submissions are the whole graded week; there is no third 25-point assignment.

1. README: user and need; how to open the .csproj/.sln/.slnx in Visual Studio; data format/source; controls; known limitations; test summary; where Evidence lives. Do not commit bin/, obj/, .vs/ or secrets.

2. Evidence contains today's brief, prototype, whiteboard/flowchart, lecture notes, peer check, tests, running/output screenshots, AI check and receipt. Keep source data under SampleData/. Store generated output under Evidence/Outputs/.

3. Use Download receipt in this book. Complete the checkboxes honestly and save the receipt in Evidence. The receipt records your selections/declarations; it does not prove attendance, app correctness or instructor acceptance.

4. Git Changes > review files > Commit All with a useful message > Push. Open the repository link in a separate browser session or with a peer and check instructor access. Private repositories require explicit instructor access.

5. Canvas > Action 9B > Start Assignment > Website URL: paste the repository URL > Submit. The repository must contain this job and its receipt; submitting a local file path is not enough. Canvas is the official record.

6. After both days: verify the two links and evidence. Reuse one useful pattern (load -> validate -> display -> export) in your next app. Do not resubmit or rewrite the two reports just to create another weekly package.

PICTURE IT

A handoff is a labelled package another person can open and verify, not a photograph of a closed box.

RUN / CHECK

The instructor can open your source, run the app, inspect evidence and find the receipt. Check Canvas shows a submission.

My observation / question / next step

Five requirements. One handoff.

Each requirement is worth 3 points. All five are required. Reuse class evidence; no duplicate reports.

R1 / Plan a different client job

Save a short brief and prototype showing File, View and Help across the top. Whiteboard the valid-row and rejected-row branches. Keep class notes and a concise Discord peer check.

R2 / Build a second blank WPF project

Create RelayManifest yourself. Open a text file, split and validate each row, show accepted records and line-number warnings, filter HIGH, and save a visible summary without changing the input.

R3 / Test and explain

Capture the menu, accepted rows, rejected-line warnings and saved summary. Test a bad number, missing field, unknown priority, all-invalid file, cancel and no-data save. Explain array, loop, TryParse and return.

R4 / Check AI and the three hats

Record a focused AI question, a test and your keep/change/reject decision (supplied-response fallback available). Explain one Visual, Logical and Data decision and when you would move from text to SQLite/MySQL.

R5 / Submit your second handoff

Commit and push this project and its Evidence, including today's receipt. Test instructor access. Submit the GitHub URL to Canvas Action 9B before today ends. Keep 9A evidence; do not rewrite it.

Handoff / receipt checklist

Use Download receipt in the online book. Enter your name and repository URL, check your work honestly, save the receipt in Evidence, commit, push and submit the URL in Canvas. The receipt is a declaration, not proof of attendance or correctness.

Evidence inventory

Brief / prototype / whiteboard or flowchart / lecture notes / Discord peer check / tests and observed results / running app and output screenshots / AI question-test-decision / receipt.

When a build fails

Read the first error, check the filename and scope, and compare the online recovery reference. A method belongs inside its class, not inside another method. Duplicate method errors mean an old handler was not replaced. Preserve a Git checkpoint before experiments.

Official references

WPF: https://learn.microsoft.com/dotnet/desktop/wpf/overview/

WPF binding: https://learn.microsoft.com/dotnet/desktop/wpf/data/

Layout panels: https://learn.microsoft.com/dotnet/desktop/wpf/controls/panel

JSON: https://jsonplaceholder.typicode.com/

JSON parsing: https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/deserialization

Flowcharts: https://app.diagrams.net/

SQLite: https://www.sqlite.org/whentouse.html

MySQL: https://dev.mysql.com/doc/refman/8.4/en/what-is-mysql.html